

Finals

Activity

NAME: Fitzgerald Ng

SECTION: SBIT-3K

DATE: 2026-03-23

Task 1: Answer the questions in short phrases:

1. When you face a large or complicated task (at school or in life), what is your usual strategy for handling it? Example: breaking it into steps, asking for help, trial and error, etc.

Reading, Analyzing, Pondering, Rereading, Trial and Error, and Solving.

2. In your own words, what do you think “divide and conquer” means?

Divide and Conquer means if the problems are large you analyze and separate it into smaller problems and start solving the smaller problems until you solve the entire problem.

3. Have you encountered any algorithm that breaks a problem into parts (e.g., Binary Search, Merge Sort)? If yes, give an example.

Yes, Binary Search and Merge Sort are the ones that I know of.

Task 2:

Match each scenario to an algorithm strategy: **Divide & Conquer**, **Decrease & Conquer**, or **Transform & Conquer**. Then explain your answer

Strategy choices:

- Divide & Conquer
- Decrease & Conquer
- Transform & Conquer

A. Finding a word inside a dictionary by repeatedly splitting the pages in half.

(1) Divide and Conquer because dictionary tends to be arranged alphabetically and so I think it is similar to problems that require binary search for efficiency.

B. Solving a maze by exploring one path at a time and reducing options as you go.

(2) Reduce and Conquer because we keep writing off path as we go, so it is best to use Reduce and Conquer.

C. Sorting a list by breaking it into small sub lists, sorting each one, then combining.

(3) Transform and Conquer because I think we are not only dividing the list but also sorting, so I think Transform and Conquer works.

Task 3: Imagine you are designing a program to search for a student's ID number in a sorted list of 10,000 IDs.

1. Which strategy do you think is more efficient:

- ☐ Checking IDs one by one
- ☐ Splitting the list repeatedly
- ☐ Reducing the list size gradually

2. Explain your reasoning using 2–3 sentences.

I think splitting the list repeatedly works best because one-by-one is $O(n)$ and reducing the list does not really make sense in this context (where do we start reducing?). Splitting works well because we could use binary search which has $O(\log(n))$ which is faster than $O(n)$ especially in the long run.


```

import java.util.Map;

public class GreedyAlgo {

    public static void main(String[] args) {
        // Column name: id, value, size, ratio
        double[][] knapsack = {{0.0, 60.0, 10.0, 6.0}, {1.0, 100.0, 20.0, 5.0},
                               {2.0, 120.0, 30.0, 4.0}};
        Map<Double, String> idName = Map.of(0.0, "A", 1.0, "B", 2.0, "C");
        double capacity = 50;
        double value = 0;

        String selectedItems = "";
        for (double[] item : knapsack) {
            if(item[2] <= capacity) {
                value += item[1];
                capacity = capacity - item[2];
                selectedItems += idName.get(item[0]) + ", ";
            }else {
                double fractionPortion = capacity/item[2];
                value += item[1] * fractionPortion;
                capacity = capacity - (item[2] * fractionPortion);
                selectedItems += String.format("%.2f", fractionPortion) +
                    " of " + idName.get(item[0]);
            }
        }

        System.out.println("The items in the knapsack are: " + selectedItems);
        System.out.println("Total Value is: " + value);
    }
}

```

Output:

```

The items in the knapsack are: A, B, 0.67 of C
Total Value is: 240.0

```